

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

INGENIERÍA MECATRÓNICA



2026-1

1MTR56

AUTOMATIZACIÓN INDUSTRIAL INTELIGENTE B

LABORATORIO 1: COMPUTADORA INDUSTRIAL BECKHOFF (IPC)

Tabla de contenido

1	Introducción	4
2	Fundamentos de computadoras industriales (IPC)	5
2.1	Conceptos básicos	5
2.2	Arquitectura y componentes	5
2.3	Comparativa entre computadoras industriales (IPC) y controladores lógicos programables (PLC)	5
2.4	Introducción a Beckhoff y EtherCAT	6
2.4.1	Beckhoff: Automatización basada en PC	6
2.4.2	EtherCAT: Protocolo de comunicación en tiempo real	6
2.5	Perspectivas futuras y aplicaciones emergentes	6
3	Programación en computadoras industriales	7
3.1	Lenguajes de programación en la industria según IEC 61131-3	7
3.1.1	Lenguaje Ladder (LD)	7
3.1.2	Texto estructurado (ST)	7
3.2	Manejo y tipos de variables	10
3.2.1	Gestión de variables	11
3.3	Funciones esenciales en la programación	11
3.3.1	Contactores: NA, NC	12
3.3.2	Bobinas: Set, Reset	12
3.3.3	Lógicas: AND, OR	12
3.3.4	Aritméticas: ADD, SUB, MUL, DIV	13
3.3.5	Comparativas: LT, LE, GT, GE	13
3.3.6	Temporizadores: TON, TOF	14
3.3.7	Contadores: CTU, CTD	14
4	Diseño y configuración de interfaces hombre-máquina (HMI)	17
4.1	Conceptos fundamentales de HMI	17
4.2	Comparativa entre HMI, sistemas SCADA y DCS	17
4.3	Mejores prácticas para el diseño de HMIs eficientes	18
4.4	Mejores prácticas para la programación: separación de la lógica de estados y la lógica de actuadores	19
5	Parte practica	21

5.1	Equipos del laboratorio	21
5.1.1	Conexiones y comunicación	21
5.1.2	Integración de entradas y salidas (E/S)	22
5.2	Experiencia 1: Semáforo (lenguaje Ladder) (3 pts)	24
5.3	Experiencia 2: Semáforo (texto estructurado) (4 pts)	27
5.4	Experiencia 3: Sistema de paletizado automático (Pick & Place) (LD) (5 pts)	29
5.5	Experiencia 4: Sistema de paletizado automático (Pick & Place) (ST) (8 pts)	32
6	Conclusiones y retroalimentación	35
7	Bibliografía	36

1 Introducción

En esta guía se presenta un laboratorio práctico cuyo objetivo es enseñar a los estudiantes el manejo de la computadora industrial Beckhoff (IPC) en un entorno de automatización.

Objetivos del laboratorio:

- Conocer los fundamentos y diferencias entre IPC y PLC.
- Aprender a programar en los lenguajes estándar definidos por IEC 61131-3 (Ladder y Structured Text).
- Diseñar y configurar interfaces HMI optimizadas para el control de procesos.
- Aplicar estos conocimientos en el desarrollo de sistemas prácticos, como semáforos y sistemas pick & place.

Conocimientos previos recomendados:

- Conceptos básicos de electrónica y control industrial.
- Familiaridad con entornos de programación y lógica de control.

2 Fundamentos de computadoras industriales (IPC)

2.1 Conceptos básicos

Las computadoras industriales (IPCs) son dispositivos de alta fiabilidad y procesamiento, diseñados para operar en entornos hostiles. A diferencia de las computadoras convencionales, los IPCs están contruidos para soportar temperaturas extremas, vibraciones, polvo y humedad, lo que los hace idóneos para aplicaciones críticas en la automatización industrial.

2.2 Arquitectura y componentes

Los IPCs se componen -de manera simplificada- de tres partes fundamentales:

- A. **Unidad Central de Procesamiento (CPU):** Basada en procesadores x86 o ARM, con capacidades multicore y soporte para virtualización, ofreciendo el rendimiento necesario para aplicaciones avanzadas.
- B. **Interfaz de Entrada/Salida (I/O):** Conecta sensores, actuadores y otros dispositivos mediante módulos digitales y analógicos, asegurando la interacción con el entorno físico.
- C. **Software:** Incluye sistemas operativos en tiempo real (RTOS) y plataformas de desarrollo como TwinCAT, que permiten la implementación de aplicaciones de control y monitoreo en tiempo real.

2.3 Comparativa entre computadoras industriales (IPC) y controladores lógicos programables (PLC)

Aunque los PLCs han sido la tecnología tradicional en la automatización, los IPCs ofrecen ventajas clave en términos de flexibilidad y capacidad de procesamiento. A continuación, se resume la comparación:

Tabla 2.1: Comparativa de PLC vs IPC. Fuente: elaboración propia.

Característica	PLC	IPC
Procesamiento	Microprocesadores dedicados	Procesadores x86/ARM, multicore
Lenguajes de Programación	IEC 61131-3 (Ladder, ST, etc.)	IEC 61131-3, C/C++, Python, MATLAB
Expansión de Hardware	Limitada a módulos específicos	Compatible con tarjetas de terceros, USB, PCIe
Conectividad	Protocolos industriales estándar	Ethernet, OPC UA, MQTT, Modbus, etc.
Aplicaciones	Control secuencial y procesos simples	Control avanzado, análisis de datos, IA
Costo	Bajo a medio	Medio a alto, según la aplicación

2.4 Introducción a Beckhoff y EtherCAT

2.4.1 Beckhoff: Automatización basada en PC

Beckhoff es un líder en tecnología de automatización conocido al integrar el poder de los IPCs con un software robusto y flexible. Su entorno TwinCAT permite la programación en tiempo real y la integración de múltiples funcionalidades (PLC, CNC, robótica) en una única plataforma.

Entre las ventajas de la tecnología Beckhoff destacan:

- Integración de hardware y software: Minimiza la necesidad de dispositivos dedicados al control.
- Modularidad y escalabilidad: Facilita la expansión del sistema mediante módulos de I/O distribuidos.
- Compatibilidad con estándares abiertos: Soporte para OPC UA, MQTT y otras tecnologías relevantes en Industria 4.0.

2.4.2 EtherCAT: Protocolo de comunicación en tiempo real

EtherCAT es un protocolo industrial basado en Ethernet, diseñado para ofrecer alta velocidad y baja latencia mediante la técnica de procesamiento en el paso (on-the-fly processing).

Características principales de EtherCAT:

- Baja latencia y alta velocidad: Ciclos de comunicación inferiores a 100 μ s.
- Topología flexible: Soporta configuraciones en línea, estrella y árbol sin degradar el rendimiento.
- Diagnóstico avanzado: Permite detectar y resolver fallos en tiempo real.

Esta tecnología es esencial en aplicaciones como robótica, sistemas CNC y líneas de producción sincronizadas, donde la precisión y la velocidad son críticas.

2.5 Perspectivas futuras y aplicaciones emergentes

El avance de los IPCs está vinculado a las tendencias de la Industria 4.0 y la digitalización. Algunas áreas de desarrollo incluyen:

- **IIoT (Internet Industrial de las Cosas):** Conexión de sensores y dispositivos a la nube para monitoreo y mantenimiento predictivo.
- **Inteligencia Artificial y Machine Learning:** Optimización de procesos mediante algoritmos de análisis de datos en tiempo real.
- **Gemelos Digitales:** Modelos virtuales que replican procesos industriales para simular y mejorar el rendimiento del sistema.

Estas innovaciones posicionan a los IPCs como elementos fundamentales en la transformación digital de la industria, permitiendo optimizar la producción, reducir costos y mejorar la eficiencia operativa.

El IEC 61131-3 estandariza la programación de PLCs, garantizando independencia de fabricantes y portabilidad.

3 Programación en computadoras industriales

3.1 Lenguajes de programación en la industria según IEC 61131-3

El estándar IEC 61131-3 define varios lenguajes de programación para automatización industrial.

Entre ellos se encuentran:

- **Ladder (LD):** Representa la lógica mediante diagramas eléctricos.
- **Texto Estructurado (ST):** Ofrece una sintaxis similar a lenguajes de programación de alto nivel.
- **Diagrama de Bloques Funcionales (FBD), Lista de Instrucciones (IL) y Diagrama de Funciones Secuenciales (SFC).**

Cada lenguaje tiene ventajas particulares, permitiendo elegir el más adecuado según la complejidad y naturaleza de la aplicación.

3.1.1 Lenguaje Ladder (LD)

También conocido de contactos o Ladder, en este lenguaje de programación las instrucciones son representadas como símbolos eléctricos. La programación Ladder permite visualizar fácilmente el sentido de circulación de la corriente a través de contactos, elementos complejos y bobinas.

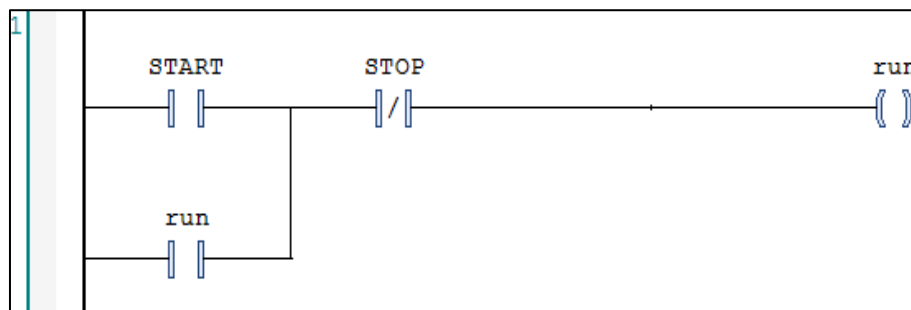


Figura 3.1: Diagrama ladder. Fuente: Elaboración propia.

3.1.2 Texto estructurado (ST)

El lenguaje de alto nivel ST (Structured Text) ofrece comandos textuales que permiten desarrollar aplicaciones bien estructuradas. Un archivo fuente ST consiste en texto continuo, organizado en secciones que representan unidades lógicas. Sus principales ventajas incluyen:

- Integración de Motion Control, PLC y funciones tecnológicas en un solo lenguaje.
- Programas estructurados con soporte para comentarios.
- Herramientas avanzadas de edición, como resaltado de sintaxis, autocompletado y sangría automática.
- Funciones de depuración intuitivas para pruebas y diagnóstico en línea, incluyendo visualización de variables y puntos de interrupción en el editor.

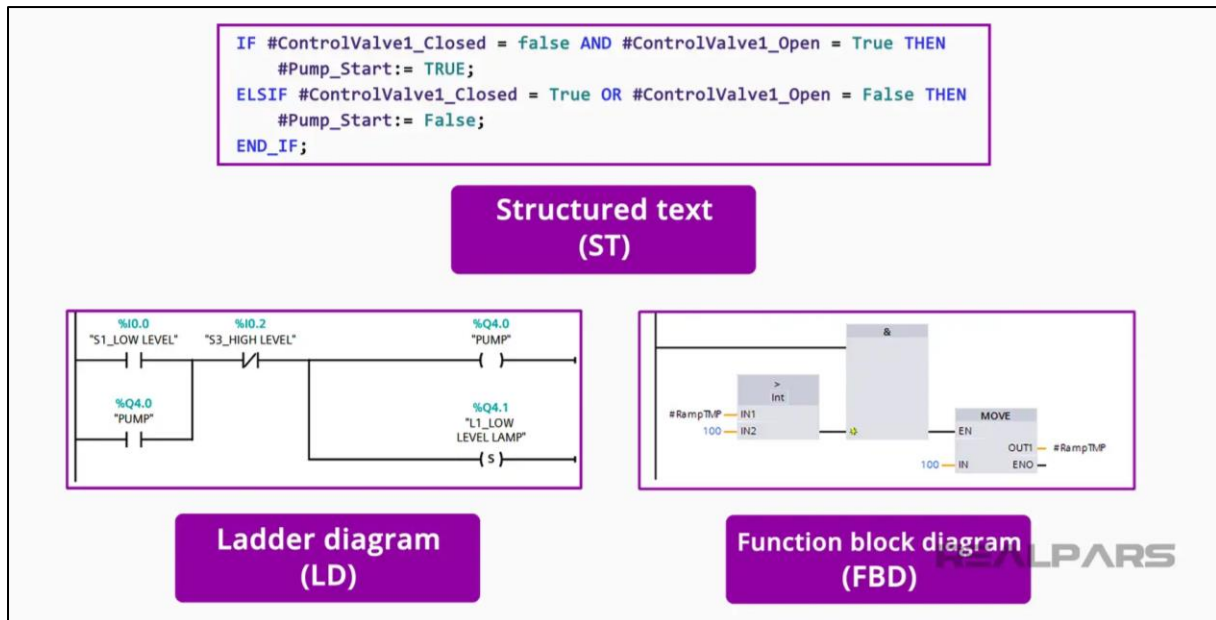


Figura 3.2: Tipos de lenguaje de programación. Fuente: Realpars.

3.1.2.1 Instrucción IF

La instrucción IF evalúa una condición expresada como un valor booleano. Si la condición es TRUE, se ejecutan las instrucciones después de THEN; en caso contrario, se evalúan secuencialmente las condiciones ELSIF. Si alguna es TRUE, su bloque correspondiente se ejecuta. Si ninguna condición se cumple, el bloque ELSE (si existe) se ejecuta. Solo una rama se ejecuta por ciclo, y los bloques ELSIF y ELSE son opcionales.

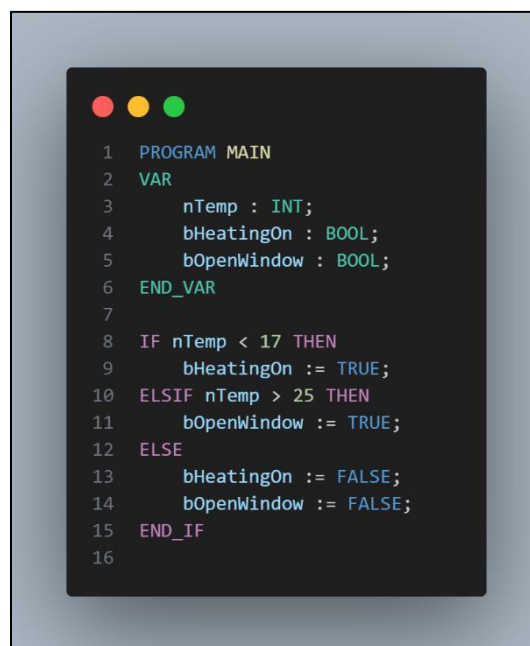
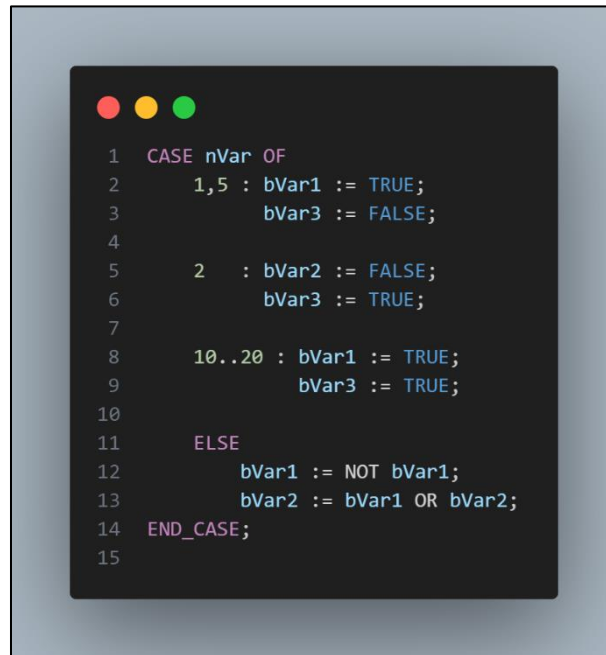


Figura 3.3: Ejemplo instrucción IF.

3.1.2.2 Instrucción CASE

La instrucción `CASE` permite evaluar múltiples condiciones sobre una misma variable, facilitando la estructuración del código. Similar al bloque `switch` en otros lenguajes, es útil para la implementación eficiente de máquinas de estados.

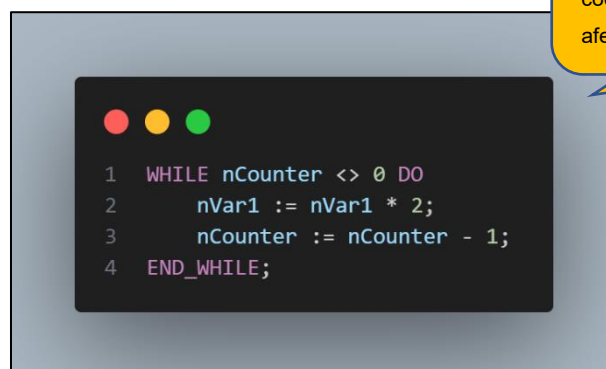


```
1 CASE nVar OF
2   1,5 : bVar1 := TRUE;
3       bVar3 := FALSE;
4
5   2   : bVar2 := FALSE;
6       bVar3 := TRUE;
7
8   10..20 : bVar1 := TRUE;
9         bVar3 := TRUE;
10
11 ELSE
12   bVar1 := NOT bVar1;
13   bVar2 := bVar1 OR bVar2;
14 END_CASE;
15
```

Figura 3.4: Instrucción CASE.

3.1.2.3 Instrucción WHILE

El bucle `WHILE`, al igual que `FOR`, ejecuta instrucciones repetidamente hasta que se cumpla una condición de finalización, definida por una expresión booleana.



```
1 WHILE nCounter <> 0 DO
2   nVar1 := nVar1 * 2;
3   nCounter := nCounter - 1;
4 END_WHILE;
```

! **Advertencia:** No utilizar durante el laboratorio. Un uso indebido de este código podría generar un bucle infinito, afectando el rendimiento del sistema.

Figura 3.5: Instrucción WHILE.

TwinCAT ejecuta las instrucciones en bucle mientras la expresión booleana evaluada sea `TRUE`. Si es `FALSE` en la primera evaluación, las instrucciones no se ejecutan. Si nunca cambia a `FALSE`, el ciclo se repite indefinidamente, generando un error de tiempo de ejecución.

3.2 Manejo y tipos de variables

La correcta declaración y gestión de variables es fundamental en la programación industrial. Según IEC 61131-3, las variables se clasifican en:

- **Básicas:** BOOL, INT, REAL, STRING, entre otras.
- **Derivadas:** ARRAY, STRUCT y ENUM, que permiten agrupar y gestionar datos.

Tipo de variables básicas:

Tabla 3.1: Tipos de variables básicas.

Tipo de Dato	Tamaño	Rango	Descripción
BOOL	1 bit	TRUE o FALSE	Representa un valor booleano, utilizado para decisiones lógicas.
INT	16 bits	-32,768 a 32,767	Entero con signo de tamaño medio, común en cálculos generales.
REAL	32 bits (IEEE 754)	-3.4×10^{-38} a $+3.4 \times 10^{38}$	Número en punto flotante, usado para operaciones con valores decimales.
STRING	Variable (hasta 255)	0 a 255 caracteres	Cadena de caracteres, útil para el manejo de texto y mensajes.

Tipo de variables derivadas:

- ✓ **ARRAY:** Colección de elementos del mismo tipo, organizados en un índice. Por ejemplo, ARRAY [1..10] OF INT define un array de 10 enteros.
- ✓ **STRUCT:** Estructura de datos que agrupa variables de diferentes tipos bajo un solo nombre.

Ejemplo de una definición de estructura denominada PolygoneLine:

```

1  TYPE PolygoInline:
2  STRUCT
3      Start : ARRAY [1..2] OF INT;
4      Point1 : ARRAY [1..2] OF INT;
5      Point2 : ARRAY [1..2] OF INT;
6      Point3 : ARRAY [1..2] OF INT;
7      Point4 : ARRAY [1..2] OF INT;
8      End : ARRAY [1..2] OF INT;
9  END_STRUCT
10 END_TYPE

```

Figura 3.6: Ejemplo de STRUCT. Fuente: Beckhoff.

- ✓ **ENUM**: Enumeración de valores, útil para definir un conjunto finito de estados o opciones.

Ejemplo:



Figura 3.7: Ejemplo de ENUM. Fuente: Beckhoff.

3.2.1 Gestión de variables

La correcta gestión de variables en sistemas IPC sigue los principios de declaración explícita, inicialización adecuada y un manejo claro del alcance de las variables. A continuación, se especifican los alcances comunes de las variables:

Tabla 3.2: Variables locales y globales.

Alcance	Descripción
Variables locales	Declaradas y utilizadas dentro de un bloque de programa o función específica. No son accesibles fuera de este, lo que previene conflictos de nombres y mejora la modularización del código.
Variables globales	Definidas en un área común y accesibles por todos los bloques y funciones del programa, permitiendo la comunicación y el intercambio de datos en todo el sistema.

3.3 Funciones esenciales en la programación

Se detallan funciones clave, como:

- Operadores Lógicos: AND, OR.
- Funciones Aritméticas: ADD, SUB, MUL, DIV.
- Comparadores: LT, LE, GT, GE.
- Temporizadores y Contadores: TON, TOF, CTU, CTD.

Cada función se explica con tablas, ejemplos y diagramas, permitiendo al estudiante comprender cómo implementar estas operaciones en un entorno real.

A continuación, se describen las funciones esenciales, categorizadas por su tipo y uso.

3.3.1 Contactores: NA, NC

Los contactores son elementos básicos en los circuitos de control. Se utilizan para abrir o cerrar un circuito bajo determinadas condiciones lógicas.

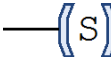
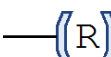
Tabla 3.3: Contactores.

Símbolo	Nombre	Descripción
	Contacto NA (Normalmente Abierto)	Se activa (cierra el circuito) cuando se recibe un "1" lógico en el elemento que representa.
	Contacto NC (Normalmente Cerrado)	Se activa (abre el circuito) cuando se recibe un "0" lógico en el elemento que representa.

3.3.2 Bobinas: Set, Reset

Las bobinas son componentes que permiten la activación o desactivación de elementos en un circuito de control.

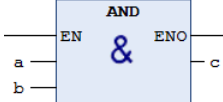
Tabla 3.4: Bobinas.

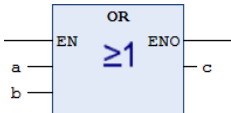
Símbolo	Nombre	Descripción
	Set	Una vez activada, la bobina permanece en este estado hasta que sea desactivada por su correspondiente bobina RESET.
	Reset	Desactiva una bobina SET previamente activada.

3.3.3 Lógicas: AND, OR

Los operadores lógicos permiten la combinación de condiciones para el control de procesos.

Tabla 3.5: Operadores lógicos.

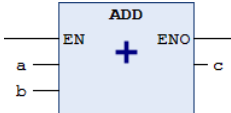
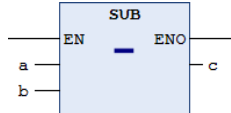
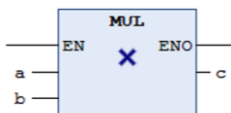
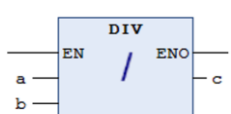
Símbolo	Nombre	Descripción
	AND	La salida c será 1 (verdadero) solo si todas las entradas (a y b) son 1 lógicos. De lo contrario, la salida será 0.

	OR	La salida c será 1 si al menos una de las entradas (a o b) es 1 lógico. Solo será 0 cuando todas las entradas sean 0.
---	----	---

3.3.4 Aritméticas: ADD, SUB, MUL, DIV

Las funciones aritméticas permiten realizar cálculos matemáticos básicos en sistemas de control.

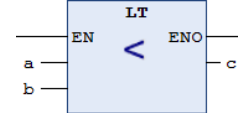
Tabla 3.6: Funciones aritméticas.

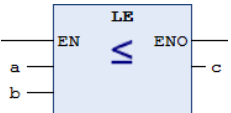
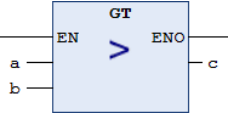
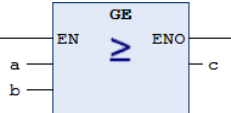
Símbolo	Nombre	Descripción
	ADD	Calcula la suma de dos valores (a + b) y devuelve el resultado c.
	SUB	Resta el valor b del valor a y devuelve el resultado c.
	MUL	Multiplica dos valores (a * b) y devuelve el resultado c.
	DIV	Divide el valor a entre el valor b y devuelve el resultado c.

3.3.5 Comparativas: LT, LE, GT, GE

Los comparadores se utilizan para comparar valores numéricos y generar una salida lógica en función del resultado.

Tabla 3.7: Comparadores.

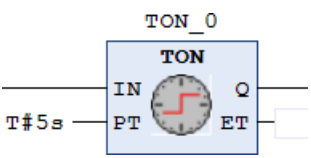
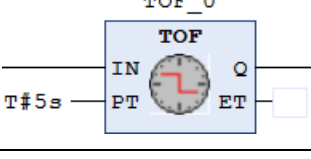
Símbolo	Nombre	Descripción
	LT (Menor que)	La salida c será VERDADERO si a es menor que b.

	LE (Menor o Igual que)	La salida c será VERDADERO si a es menor o igual que b.
	GT (Mayor que)	La salida c será VERDADERO si a es mayor que b.
	GE (Mayor o Igual que)	La salida c será VERDADERO si a es mayor o igual que b.

3.3.6 Temporizadores: TON, TOF

Los temporizadores controlan el tiempo de activación o desactivación de una salida.

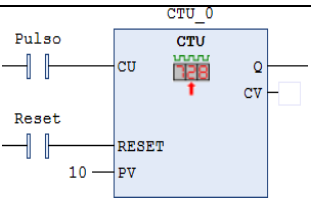
Tabla 3.8: Temporizadores.

Símbolo	Nombre	Descripción
	TON (Temporizador a la Conexión)	Activa una salida (Q) después de que transcurra un tiempo PT determinado desde que la entrada (IN) se activó.
	TOF (Temporizador a la Desconexión)	Mantiene una salida (Q) activada durante un tiempo PT después de que la entrada (IN) se haya desactivado.

3.3.7 Contadores: CTU, CTD

os contadores incrementan o decrementan su valor en función de entradas específicas, útil en procesos que requieren control de cantidades.

Tabla 3.9: Contadores.

Símbolo	Nombre	Descripción
	CTU	Incrementa el valor del contador (CV) cada vez que la entrada de conteo (CU) detecta un flanco ascendente. El contador se resetea cuando la entrada RESET es verdadera.

	CTD	Decrementa el valor del contador (CV) cada vez que la entrada de conteo (CD) detecta un flanco ascendente. El contador se inicializa al valor máximo (PV) cuando la entrada LOAD es verdadera.
--	-----	---

Contadores en ST:

Para utilizar contadores en lenguaje Structured Text (ST), primero debes declarar la variable correspondiente en la sección de declaración de variables. Por ejemplo, para un contador ascendente, puedes declarar una variable llamada CTU_0 de tipo CTU.

```

2
3  VAR
4      CTU_0: CTU;
5  END_VAR
6
7
8  CTU_0 (
9      CU:= ,
10     RESET:= ,
11     PV:= ,
12     Q=> ,
13     CV=> );

```

Figura 3.8: Contadores en ST.

En la sección de programación, se debe instanciar el contador CTU_0 y configurar sus componentes. Recuerda lo siguiente al trabajar con contadores en ST:

- **Entradas:** Las variables que se encuentran en el lado izquierdo de un bloque de funciones en Ladder son entradas. En ST, estas variables deben ser asignadas utilizando el operador ":=".
- **Salidas:** Las variables en el lado derecho del bloque de funciones en Ladder son salidas. En ST, estas variables son de solo lectura, y si deseas almacenar el valor en otra variable, debes asignarlo utilizando el operador "=>".

Este enfoque permite replicar la funcionalidad de un contador en Ladder dentro de un programa en ST, facilitando su integración en sistemas más complejos.

Para agregar cualquier función en Structured Text (ST) de manera sencilla en TwinCAT, sigue estos pasos:

1. Hacer clic derecho en la sección de programación y abrir el Input Assistant.

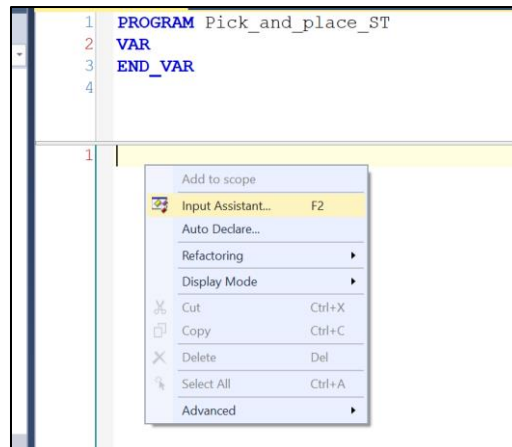


Figura 3.9: Input Assistant.

2. Seleccionar la función deseada:

- Aparecerá una ventana que muestra todas las librerías y funcionalidades disponibles.
- Haz clic en "Function Blocks" para desplegar las opciones y selecciona la carpeta "Counter" para ver los contadores disponibles.
- Para otras funciones, como temporizadores o detectores de flancos, sigue el mismo procedimiento explorando las carpetas correspondientes dentro del **Input Assistant**.

3. Autodeclaración de la función:

- Una vez seleccionada la función deseada, se abrirá una ventana de autodeclare
- Aquí, podrás asignar un nombre a la instancia del bloque de función, como por ejemplo un contador, y finalizar la inserción.

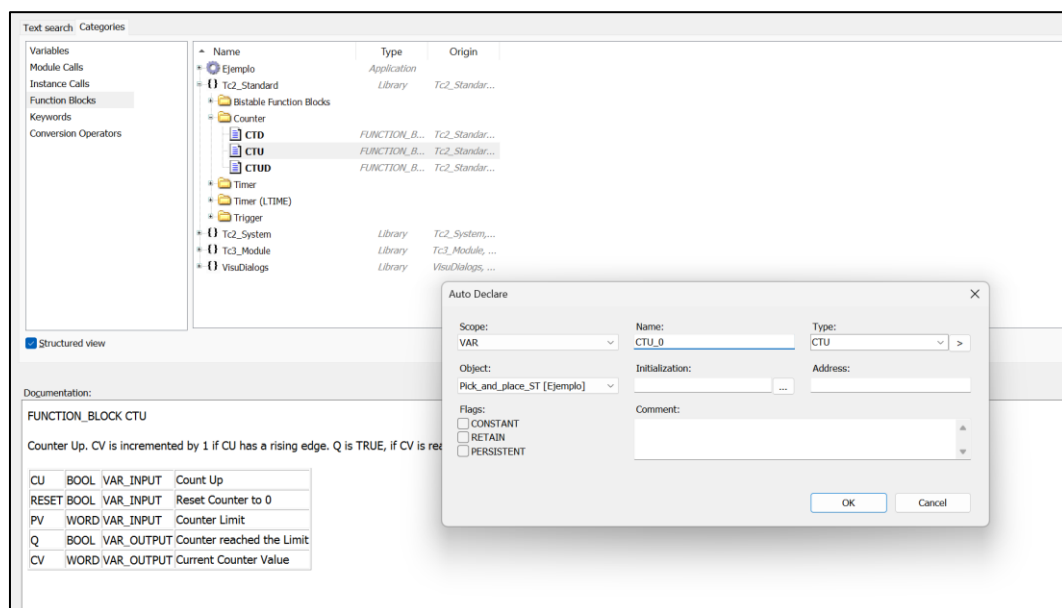


Figura 3.10: Autodeclare de funciones.

4 Diseño y configuración de interfaces hombre-máquina (HMI)

4.1 Conceptos fundamentales de HMI

El HMI es la herramienta de interacción entre el operador y el sistema de control. Un buen diseño debe garantizar:

- **Claridad visual:** Uso de paleta de colores restringida, con colores brillantes solo para alarmas o eventos críticos.
- **Uniformidad tipográfica:** Fuentes legibles y consistentes que faciliten la lectura.
- **Interactividad y accesibilidad:** Hipervínculos, botones y etiquetas descriptivas en figuras para cumplir con estándares de accesibilidad.

4.2 Comparativa entre HMI, sistemas SCADA y DCS

Los sistemas HMI, SCADA y DCS son tecnologías clave en la automatización industrial, pero difieren en propósito, escala y complejidad.

Definiciones breves:

- **HMI (Interfaz Hombre-Máquina):**

Interfaz visual para control local y monitoreo inmediato de dispositivos individuales.

Ejemplo: Pantalla táctil en una máquina prensa.

- **SCADA (Supervisory Control and Data Acquisition):**

Sistema para supervisar y controlar procesos distribuidos, integrando la adquisición de datos y gestión de alarmas de forma remota.

Ejemplo: Monitorización de una red eléctrica en una planta.

- **DCS (Distributed Control System):**

Sistema para el control en tiempo real de procesos complejos, operando de forma descentralizada con múltiples controladores.

Ejemplo: Control en tiempo real de una refinería de petróleo.

La tabla a continuación destaca las diferencias esenciales, incluyendo ejemplos de uso:

Tabla 4.1: Comparativa entre HMI, SCADAS y DCS. Fuente: elaboración propia.

Característica	HMI	SCADA	DCS
Propósito	Control local y visualización inmediata.	Supervisión y control remoto de procesos distribuidos.	Control automatizado y en tiempo real de procesos complejos.
Escalabilidad	Limitada (ideal para equipos individuales).	Alta, adecuada para supervisar plantas completas.	Muy alta, ideal para industrias con procesos críticos y continuos.
Arquitectura	Basada en un dispositivo o panel central.	Centralizada, con acceso remoto a múltiples puntos.	Descentralizada, con múltiples controladores distribuidos.
Interacción	Operación directa y sencilla.	Gestión continua de datos y alarmas.	Ajuste automático y sofisticado de parámetros operativos.
Aplicaciones Comunes	Máquinas individuales (ej. prensas, líneas de ensamblaje).	Redes de distribución, plantas de tratamiento, instalaciones energéticas.	Procesos industriales críticos (refinerías, plantas petroquímicas).
¿Cuándo Usarlo?	Cuando se necesita una interfaz local y visual sencilla.	Para supervisar y controlar procesos en diversas ubicaciones.	En entornos que requieren control distribuido y automatización avanzada.

4.3 Mejores prácticas para el diseño de HMIs eficientes

Para asegurar un diseño de HMI que cumpla con los más altos estándares industriales, se recomienda:

- **Paleta de colores:**

Usa colores neutros para la mayor parte de la interfaz y colores vivos solo para alarmas y situaciones críticas. Esto ayuda a que la información importante destaque y evita que la pantalla se vea saturada.

- **Diseño visual eficaz:**

Evita elementos innecesarios y animaciones que distraigan. La pantalla debe ser limpia, para que el operador pueda concentrarse en lo esencial.

- **Uniformidad tipográfica:**

Utiliza siempre la misma fuente y tamaño en toda la interfaz para facilitar la lectura. Una tipografía consistente mejora la claridad de la información.

- **Diseño inclusivo:**

Asegúrate de que la HMI sea accesible para todos, cumpliendo con normas ergonómicas y de accesibilidad. Por ejemplo, incluye descripciones en imágenes y gráficos para quienes lo necesiten.



Figura 4.1: Recomendación de paleta de colores. Fuente: Realpars.

4.4 Mejores prácticas para la programación: separación de la lógica de estados y la lógica de actuadores

Según *PLC Controls with Structured Text (ST), V3: IEC 61131-3 and best practice ST-programming*, se recomienda mantener una **máquina de estados** (transiciones, temporizadores, condiciones) separada de la **lógica que actualiza las salidas físicas** (actuadores/luces). La idea es que el control decide qué estado debe estar, y un bloque independiente traduce ese estado en acciones sobre las E/S.

¿Por qué?

La separación evita efectos secundarios dentro de las transiciones, hace el comportamiento más determinista y comprensible, y facilita pruebas, simulación y mantenimiento. En vez de activar salidas en distintas ramas de la lógica de cambio de estado, las salidas se calculan *en función* del estado actual.

Beneficios principales

- **Claridad y legibilidad:** separa decisión (estado) de acción (salida), facilitando la lectura del código.
- **Menos efectos secundarios:** las transiciones no contienen asignaciones dispersas a E/S que puedan provocar inconsistencias.
- **Determinismo temporal:** evita que la E/S cambie en momentos inesperados durante la evaluación de la máquina.
- **Facilidad para pruebas y simulación:** puedes simular estados sin ejecutar la lógica de transición completa.
- **Reutilización y extensibilidad:** la lógica de salida puede reutilizarse con otra máquina de estados o ajustarse sin tocar las transiciones.
- **Mantenibilidad:** cambios en la política de salidas se concentran en un único bloque.

Patrón recomendado (estructura)**A. Bloque Estado / Control**

- Implementa la máquina de estados (CASE/IF), temporizadores y asigna EstadoActual.
- No realizar asignaciones directas a actuadores aquí (salvo banderas internas necesarias).

B. Bloque Salidas / Actuadores

- Un bloque independiente lee EstadoActual y asigna todas las salidas físicas en forma declarativa (por ejemplo con expresiones booleanas).

C. Bloque I/O físico (opcional)

- Traduce variables internas a variables mapeadas al hardware (y viceversa para entradas).

Checklist rápido al aplicar esta práctica

- ¿Todas las transiciones sólo actualizan la variable de estado y no las E/S?
- ¿Las salidas son el resultado directo de EstadoActual (o de combinaciones de estado + flags), evaluadas en un único lugar?
- ¿Se pueden simular estados sin depender de temporizadores físicos?
- ¿Los cambios futuros de la lógica de salidas se limitan a un único bloque?

5 Parte practica

5.1 Equipos del laboratorio

En este laboratorio utilizaremos la computadora industrial **Beckhoff CP6606**, un dispositivo integral que combina un IPC (Industrial PC) y una HMI (Interfaz Hombre-Máquina) en una sola unidad. Este equipo está diseñado para entornos de automatización, ofreciendo una solución compacta y de alto rendimiento.

5.1.1 Conexiones y comunicación

El CP6606 dispone de dos puertos RJ45 que permiten establecer comunicaciones esenciales para el sistema:

- **Puerto Ethernet:** Se utiliza para la comunicación en red a través del protocolo TCP/IP, facilitando la conexión con el sistema de control y la integración en entornos industriales.
- **Puerto EtherCAT:** Dedicado a la comunicación en tiempo real con los módulos de entrada y salida (E/S), utilizando el protocolo EtherCAT, el cual garantiza una transmisión determinista y de alta velocidad, fundamental para aplicaciones de control.

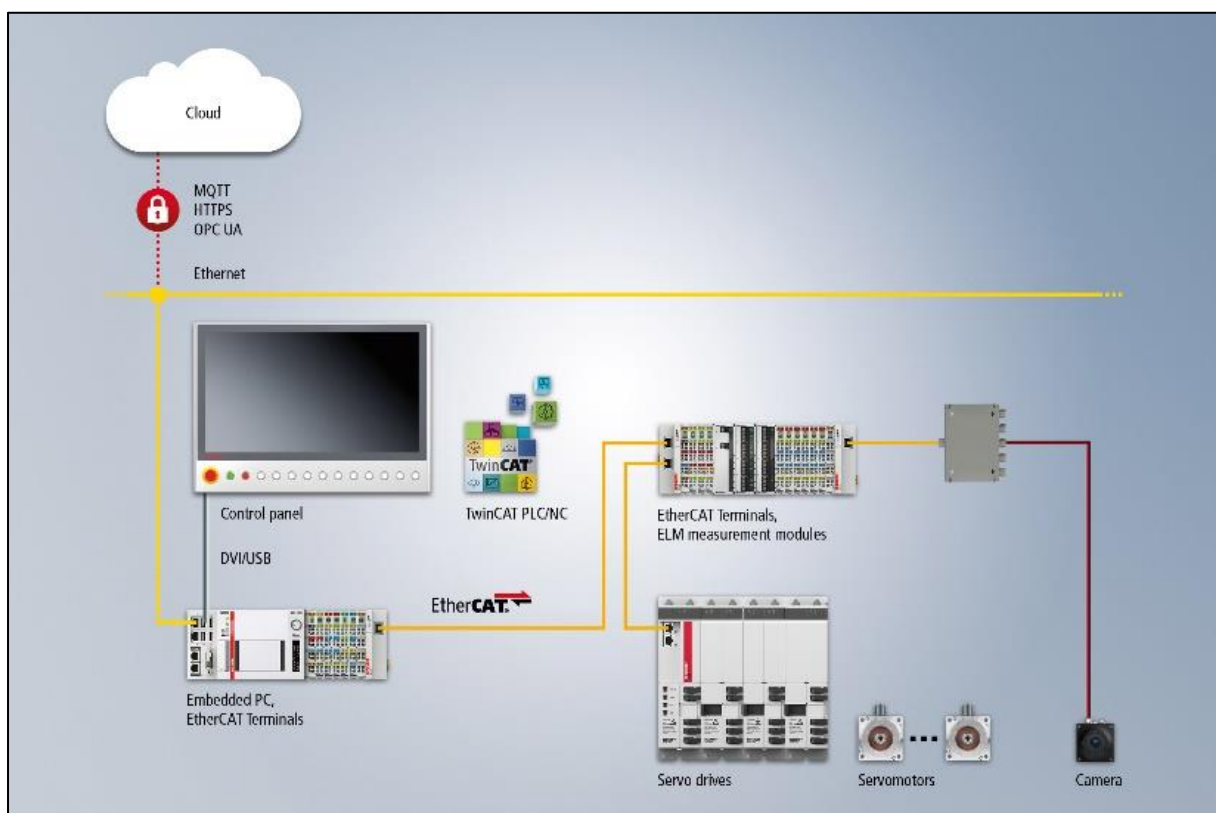


Figura 5.1: Ejemplo de red EtherCAT.

5.1.2 Integración de entradas y salidas (E/S)

A través del puerto EtherCAT, el panel PC **CP6606** se conecta al módulo **EK1100**, que actúa como acoplador **EtherCAT**. Este dispositivo es clave para ampliar las capacidades del sistema, ya que permite integrar múltiples tarjetas de E/S. Los módulos que se emplean en este laboratorio son:

- **EL1008**: Módulo de 8 entradas digitales, ideal para captar señales de sensores o dispositivos de conmutación.
- **EL2008**: Módulo de 8 salidas digitales, empleado para controlar actuadores y otros dispositivos que operan en dos estados (encendido/apagado).
- **EL3058**: Módulo de 8 entradas analógicas, diseñado para medir variables continuas como temperatura, presión o nivel.
- **EL4028**: Módulo de 8 salidas analógicas, utilizado para controlar actuadores que requieren señales variables.
- **EL6631**: Módulo especializado para comunicación **PROFINET**.

CP6606: Es un Panel PC integrado que combina una computadora industrial con una pantalla HMI en una única unidad. Está diseñado para instalarse en la parte frontal de un armario o caja de control, y es adecuado para diversas aplicaciones en la construcción de máquinas e ingeniería de plantas. El CP6606 puede operar como un controlador autónomo con el software de automatización TwinCAT sobre Windows Embedded Compact 7, o como una pantalla de escritorio remoto.



Figura 5.2: Panel PC CP6606.



Figura 5.3: Acoplador EtherCAT.



Figura 5.4: Módulos de E/S de Beckhoff integrados al acoplador EK1100.



Figura 5.5: Ejemplo de conexiones de red EtherCAT.

! Importante!:

Durante el laboratorio se implementarán cuatro pantallas, por lo que es obligatorio contar con cuatro programas (**POUs**) independientes y cuatro interfaces HMI para obtener el puntaje completo.

- ✓ Cada HMI debe incluir un **header** y un **footer**, con navegación mediante el **footer**.
- ✓ Es **obligatorio** seguir las recomendaciones de diseño y respetar la paleta de colores establecida.
- ✓ Al presionar un botón, este **debe cambiar de color** si la variable asociada cambia de estado, proporcionando una confirmación visual.
- ✓ La pantalla debe mostrar claramente todos los valores necesarios para la operación del sistema.

5.2 Experiencia 1: Semáforo (lenguaje Ladder) (3 pts)

Objetivo:

- Desarrollar un sistema de semáforo que alterne entre los colores rojo (5 s), amarillo (3 s) y verde (4 s).

Pasos:

- Configurar el sistema para iniciar mediante un botón "Start" y detenerlo con "Stop".
- Programar la lógica en Ladder, comentando cada sección para explicar el flujo de operación.
- Vincular tres salidas físicas del IPC a una visualización en el HMI que muestre los estados de cada color.

Salidas:

- HMI:** Visualización de los estados del semáforo a través de 3 pilotos luminosos.
- Salidas físicas:** 3 salidas físicas del IPC para activar los colores del semáforo.

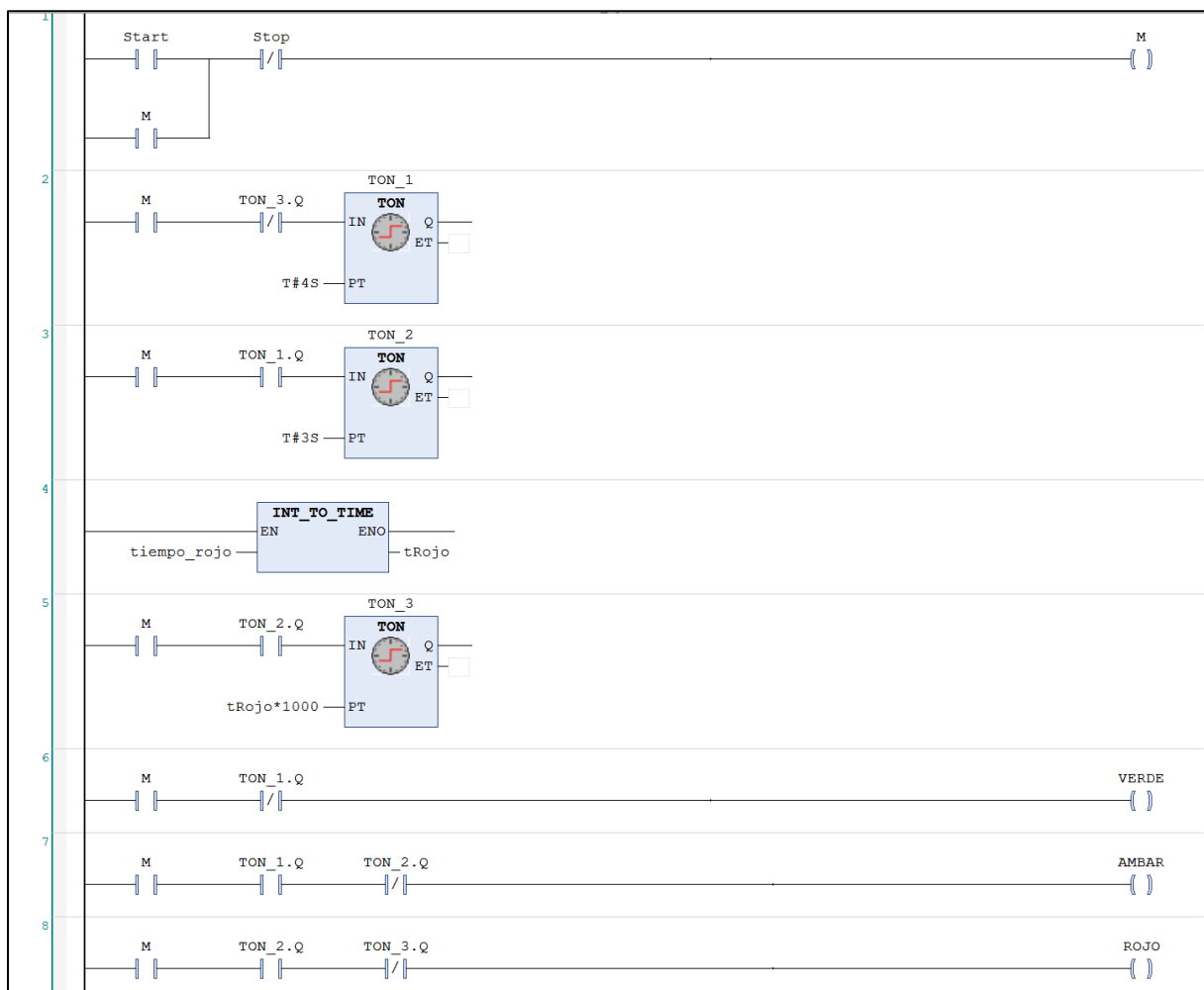


Figura 5.6: Programa en ladder.

Interfaz HMI

Se solicita implementar una interfaz HMI que incluya las siguientes secciones:

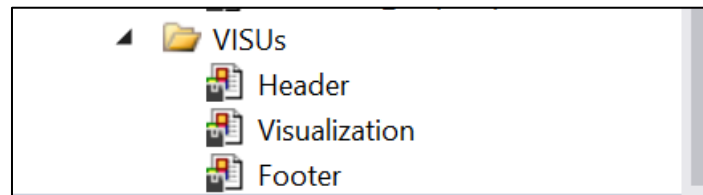


Figura 5.7: VISUs creados.

- Header:
 - Dimensiones: 800 x 50 píxeles.
 - Estructura: Dividido de forma que permita mostrar información relevante.

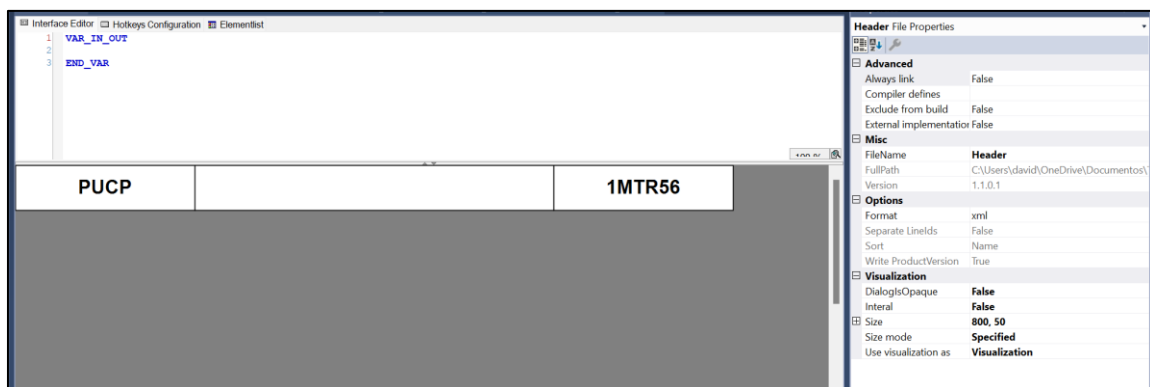


Figura 5.8: Header.

- Footer:
 - Dimensiones: 800 x 50 píxeles.
 - Contenido: Deben incluirse 4 botones de 200 x 50 píxeles cada uno.
 - El primer botón debe dirigir a la pantalla principal.
 - El segundo botón debe dirigir a una pantalla secundaria.



Figura 5.9: Footer.

Instrucciones

- Utilice la herramienta "Frame" para integrar el header y footer en las visualizaciones. Esto permitirá que cualquier modificación en el header o footer se actualice automáticamente en todas las ventanas donde se utilicen.

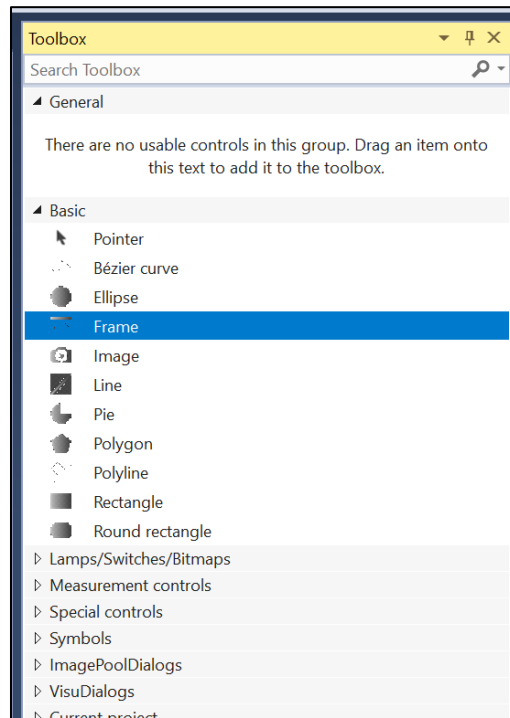


Figura 5.10: Frame.

- Añada tres lámparas en la visualización para representar los estados del semáforo.
- Agregue dos botones configurados en modo toggle para iniciar (Start) y detener (Stop) el sistema.
- Implemente un rectángulo en la HMI que permita modificar el tiempo de la luz roja, mostrando el texto: "**Tiempo luz roja: %i**", vinculado a la variable correspondiente.
- Asegure la correcta vinculación de todas las variables necesarias para el funcionamiento del sistema.



Figura 5.11: HMI.

5.3 Experiencia 2: Semáforo (texto estructurado) (4 pts)

Objetivo:

- Implementar la misma lógica del semáforo utilizando el lenguaje Structured Text.

Pasos:

1. **Crear** un nuevo POU en ST.
2. Programar la secuencia de tiempos (rojo 5 s, amarillo 3 s, verde 4 s) e integrar la funcionalidad de inicio/parada.
3. Adaptar el HMI para reflejar la nueva configuración, vinculando las variables correspondientes.

Salidas:

- **HMI:** Visualización de los estados del semáforo mediante 3 pilotos luminosos.
- **IPC:** Control de 3 salidas físicas para activar los colores del semáforo.

Tareas por realizar

1. Crear un nuevo POU en Lenguaje ST (1 pt)
 - a. Desarrolle el programa del semáforo utilizando el Lenguaje de Texto Estructurado (ST) en TwinCAT.

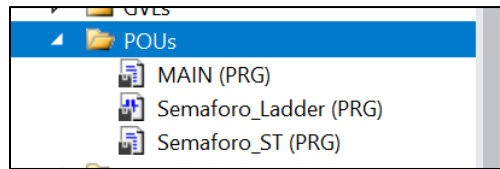


Figura 5.12: POU's desarrollados.

- b. Implemente la lógica de cambio de colores con los tiempos especificados.

```

1  (* Lógica de enclavamiento *)
2  SemaforoActivo := (SemaforoActivo OR StartButton) AND NOT StopButton;
3
4  (* Conversión INT_TO_TIME *)
5  TiempoRojo := INT_TO_TIME(iTiempoRojo)*1000;
6
7  (* Lógica del semáforo *)
8  IF SemaforoActivo THEN
9      CASE EstadoActual OF
10         0:
11             Temporizador(IN := TRUE, PT := TiempoRojo);
12             IF Temporizador.Q THEN
13                 EstadoActual := 1;
14                 Temporizador(IN := FALSE);
15             END_IF
16         1:
17             Temporizador(IN := TRUE, PT := TiempoVerde);
18             IF Temporizador.Q THEN
19                 EstadoActual := 2;
20                 Temporizador(IN := FALSE);
21             END_IF
22         2:
23             Temporizador(IN := TRUE, PT := TiempoAmarillo);
24             IF Temporizador.Q THEN
25                 EstadoActual := 0;
26                 Temporizador(IN := FALSE);
27             END_IF
28         END_CASE
29     END_IF
30
31  (* Salidas *)
32  RedLight := SemaforoActivo AND (EstadoActual = 0);
33  GreenLight := SemaforoActivo AND (EstadoActual = 1);
34  YellowLight := SemaforoActivo AND (EstadoActual = 2);
35

```

Figura 5.13: Programa en texto estructurado.

2. Modificar el HMI: (1 pt)

a. Header y Footer:

- Ajuste el encabezado y pie de página de la interfaz HMI para adaptarlos a la nueva lógica del semáforo, asegurando que reflejen la configuración actualizada de tiempos.

b. Ventana Secundaria:

- En la ventana secundaria, actualice la visualización para mostrar el nuevo semáforo con la lógica de tiempos modificada.

3. Implementar en el HMI: (2 pt)

- Agregue tres lámparas para indicar los colores del semáforo.
- Incorpore botones *Start* y *Stop*, configurados en modo *toggle*.
- Integre un rectángulo que permita modificar el tiempo de la luz roja, mostrando “*Tiempo luz roja: %i*” y vinculado a la variable correspondiente.
- Verifique la correcta vinculación de todas las variables para garantizar el funcionamiento del semáforo.



Figura 5.14: HMI- pantalla secundaria.

5.4 Experiencia 3: Sistema de paletizado automático (Pick & Place) (LD) (5 pts)

En una planta de producción de envases plásticos en Lima, se utiliza una estación de paletizado que coloca los envases fabricados dentro de cajas. El sistema cuenta con dos fajas transportadoras:

1. **Faja de Suministro B:** Transporta continuamente los envases producidos hasta que alcanzan un tope.

2. **Faja de Cajas A:** Transporta las cajas vacías hasta el punto A, donde están listas para ser llenadas.

El sistema de Pick and Place se compone de los siguientes actuadores:

- Cilindro X:
 - Posición sin accionar: en el punto A.
 - Posición accionada: se desplaza hasta el punto B.
- Cilindro Z:
 - Posición sin accionar: levantado.
 - Posición accionada: baja para recoger los envases.
- Succionadores C:
 - Accionados: sujetan los envases.
 - Sin accionar: sueltan los envases.

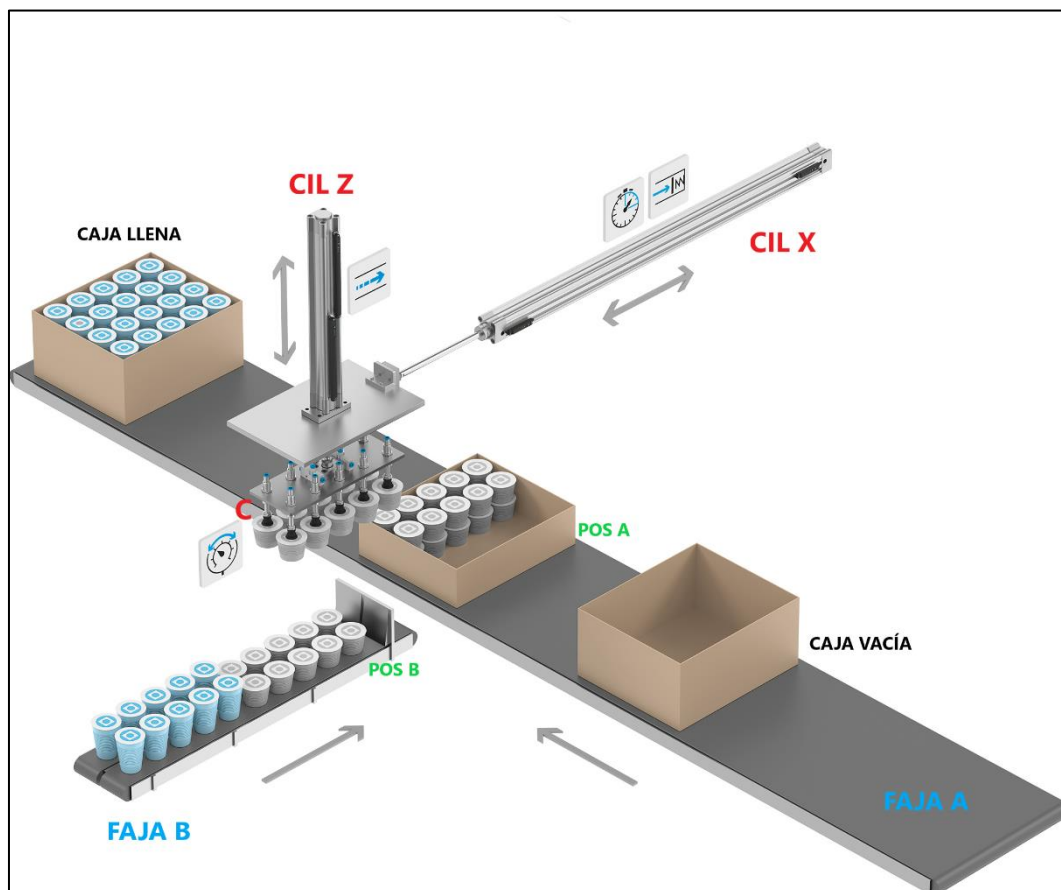


Figura 5.15: Sistema pick & place.

Secuencia Operativa:

Cada vez que se presiona un botón de inicio (start), se debe ejecutar la siguiente secuencia de pick and place:

1. Extender el Cilindro X al punto B.
2. Después de 2 segundos, extender el Cilindro Z para bajar y recoger los envases.
3. Activar los Succionadores C para sujetar los envases (después de 2 segundos).
4. Retractor el Cilindro Z a su posición inicial (después de 2 segundos).
5. Retractor el Cilindro X al punto A (después de 2 segundos).
6. Extender nuevamente el Cilindro Z (después de 2 segundos).
7. Desactivar los Succionadores C para soltar los envases.
8. Retractor el Cilindro Z para regresar a la posición inicial y esperar la siguiente secuencia.

Además, el sistema debe incluir un botón de parada (stop) que interrumpa la secuencia en cualquier momento.

- a) Elabore y desarrolle un programa en Ladder usando TwinCAT3 para controlar esta secuencia. (3 pts)
- b) Diseñe un HMI que muestre los estados de los actuadores (Cilindro X, Cilindro Z y Succionadores C) cambiando los colores de lámparas o rectángulos como se muestran en la imagen, impléméntelo en la tercera ventana del HMI y simule el programa (2 pts)

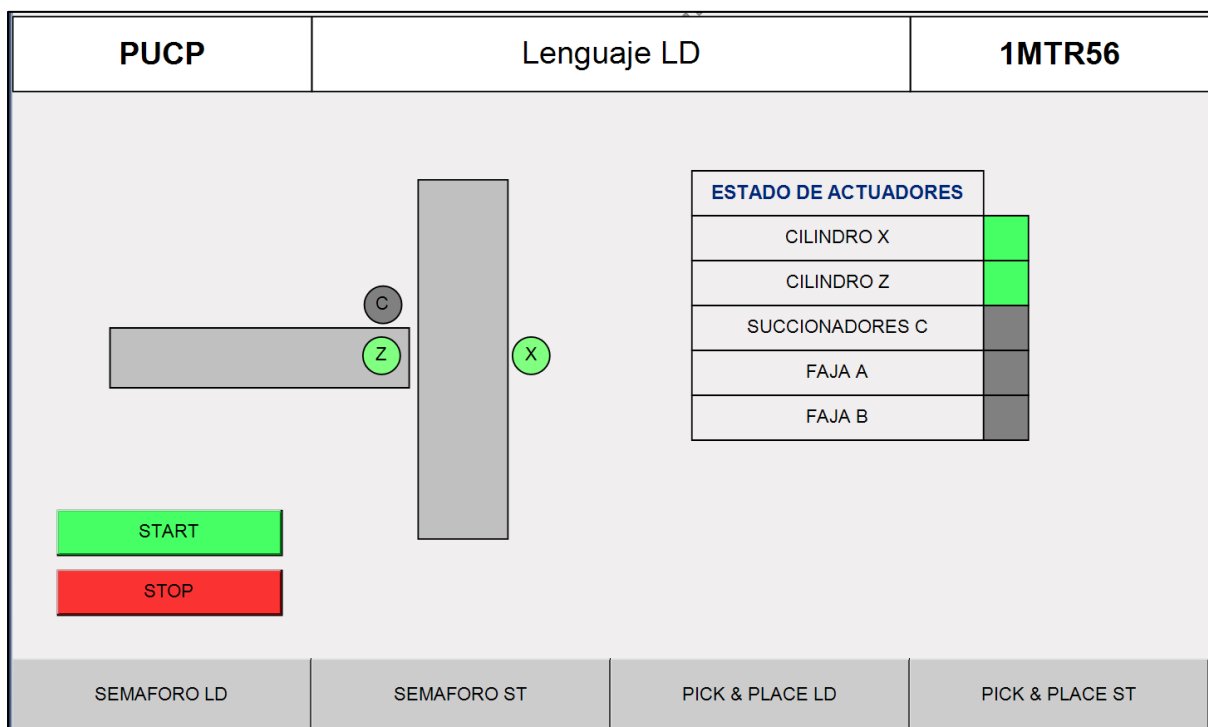


Figura 5.16: Ejemplo de HMI.

5.5 Experiencia 4: Sistema de paletizado automático (Pick & Place) (ST) (8 pts)

Continuando con el sistema de paletizado de envases plásticos en una planta de producción en Lima, ahora se introducirán nuevas funcionalidades para optimizar el proceso. El sistema actual cuenta con dos fajas transportadoras (A y B) y un sistema de pick and place que utiliza cilindros neumáticos (X y Z) y succionadores de accionamiento neumático (C).

Nuevas consideraciones:

- Ciclos de paletizado:** El proceso ahora debe manejar múltiples ciclos para llenar cajas en dos niveles (nivel izquierdo y derecho) antes de pasar a una nueva caja.
- Activación de fajas transportadoras:** Se requiere controlar el movimiento de las fajas transportadoras en momentos específicos de la secuencia.
- Programación en texto estructurado:** La lógica del sistema debe implementarse en Structured Text (ST) usando TwinCAT3.

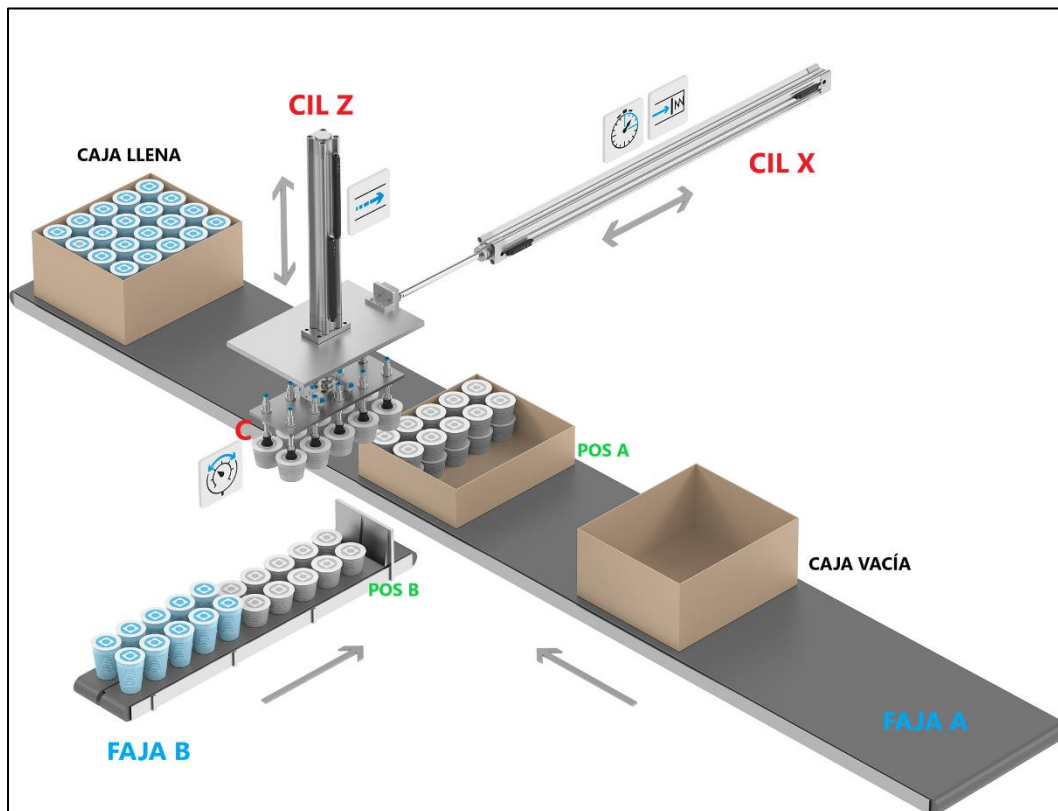


Figura 5.17: Sistema pick & place.

Secuencia Operativa:

Cada vez que se presiona un botón de inicio (start), se debe ejecutar la siguiente secuencia de pick and place:

1. **Faja Transportadora B:** Activar durante 2 segundos para mover los envases a la posición de recojo (considere que no habrá problemas de apilamiento).
2. **Cilindro X:** Extender el cilindro al punto B.
3. **Cilindro Z:** Después de 2 segundos, extender para bajar y recoger los envases.
4. **Succionadores C:** Después de 2 segundos, activar para sujetar los envases.
5. **Cilindro Z:** Después de 2 segundos, retractar a su posición inicial.
6. **Cilindro X:** Después de 2 segundos, retractar al punto A.
7. **Cilindro Z:** Después de 2 segundos, extender nuevamente el cilindro.
8. **Succionadores C:** Desactivar para soltar los envases.
9. **Cilindro Z:** Retractor para regresar a la posición inicial y esperar la siguiente secuencia.

Sobre la secuencia del llenado de cajas:

- a) Lado izquierdo:
 - Realizar dos ciclos de la secuencia para apilar los envases en el nivel 1 y 2 del lado izquierdo de la caja.
 - Activar la Faja Transportadora A durante 2 segundos para posicionar la caja para el llenado del lado derecho.
- b) Lado izquierdo:
 - Realizar dos ciclos adicionales para completar el llenado en el lado derecho.
 - Después de llenar ambos lados, activar la Faja Transportadora A durante 5 segundos para mover la caja llena y posicionar una nueva caja.

Contador de cajas:

- Una vez que una caja se haya llenado correctamente (completando los cuatro ciclos de paletizado), se debe incrementar un contador en el HMI que registre el número total de cajas llenas.
- El HMI debe incluir un botón para reiniciar el contador, pero no se debe permitir modificar manualmente el número total de cajas llenas.

Botón de Parada (Stop):

- El sistema debe incluir un botón de parada que permita interrumpir la secuencia en cualquier momento.

Tareas:

- a) Elabore y desarrolle un programa en Structured Text (ST) usando TwinCAT3 para controlar la secuencia descrita. (3.5 pts)
- b) Diseñe un HMI que muestre los estados de los actuadores (Cilindro X, Cilindro Z, Succionadores C), implemente el diseño en la cuarta ventana del HMI, y vincule las variables físicas con las salidas digitales correspondientes en las tarjetas físicas. (3.5 pts)

- c) Añada el visualization Manager > Target Visualization y cargue el programa a la computadora industrial BECKHOFF. (1 pt)

! **Importante:** Solicitar ayuda a su jefe de laboratorio para cargar el programa a la computadora industrial.

Ejemplo de HMI:

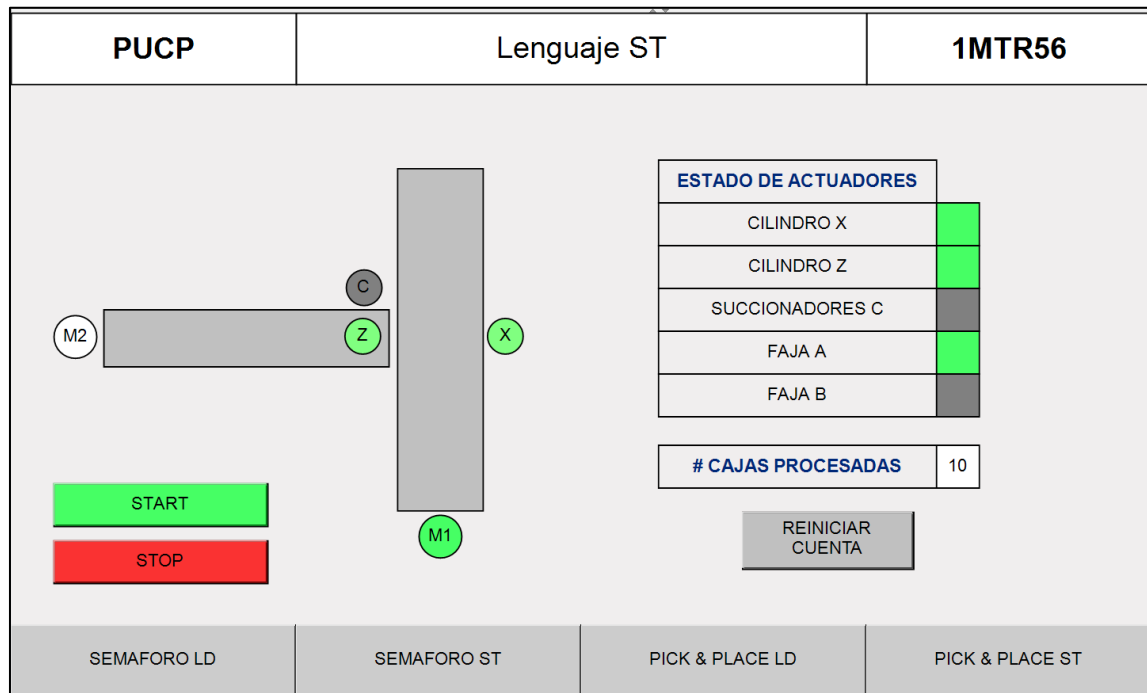


Figura 5.18. Ejemplo de HMI ST.

6 Conclusiones y retroalimentación

Lo aprendido:

- Se refuerzan los conceptos teóricos y prácticos en automatización industrial, integrando conocimientos sobre IPCs, programación (Ladder y ST) y diseño de interfaces HMI.
- La experiencia práctica permite a los estudiantes aplicar los conceptos en situaciones reales, fomentando el análisis crítico y la solución de problemas.

Recomendaciones para el estudiante:

- Revisar y analizar cada paso de la secuencia operativa, asegurándose de comprender la lógica detrás de cada función.
- Utilizar los diagramas de flujo y comentarios en el código como guía para futuros proyectos de automatización.

7 Bibliografía

LinkedIn. (2021). *TwinCAT & virtualization*. LinkedIn. <https://www.linkedin.com/pulse/twincat-virtualization-jakob-sagatowski>

Beckhoff. (2021). *Beckhoff Information System - English*. Beckhoff.
https://infosys.beckhoff.com/english.php?content=../content/1033/tc3_licensing/81064795832186251.html&id=288384927272963155

RealPars. (n.d.). *PLC vs. PC: What's the difference and which should you choose?* RealPars Blog.
<https://www.realpars.com/blog/plc-vs-pc>

Beckhoff. (n.d.). *Automation products*. Beckhoff Automation. <https://www.beckhoff.com/en-us/products/automation/>

Antonsen, T. M. (2020). *PLC controls with structured text (ST), V3: IEC 61131-3 and best practice ST-programming*. BoD – Books on Demand. ISBN 9788743015543.